

DOSSIER PROJET

Cursus Data Engineer

RNCP38919

Projet FanFlight

Plateforme de recherche de vols pour la Coupe du Monde 2026

Hugo PRADIER

Projet réalisé avec Hugo Jacquit, Daniel Kouacou et Emy Regna

Promotion 2025–2026

Liora – Certification DataScientest

31 juillet 2026

Remerciements

Je tiens à exprimer ma sincère gratitude envers tous ceux qui ont contribué à la réussite du projet FanFlight et à l'accomplissement de ma formation Data Engineer chez Liora, certifiée par DataScientest.

Mes remerciements s'adressent en premier lieu à Dan Cohen, notre mentor projet, pour son accompagnement, sa disponibilité et ses conseils techniques qui nous ont permis de faire les bons choix d'architecture au bon moment, en particulier sur les questions de coût et de robustesse du pipeline de données.

Je remercie également l'équipe pédagogique de Liora et de DataScientest pour la qualité de la formation et son accompagnement tout au long du cursus Data Engineer (RNCP38919).

Ma gratitude va tout particulièrement à mes coéquipiers, Hugo Jacquit, Daniel Kouacou et Emy Regna, avec qui ce projet a été mené du premier commit jusqu'à la mise en production. Malgré une équipe répartie sur toute la France et des emplois du temps contraints par nos alternances respectives, nous avons su maintenir une dynamique de travail collective sur la durée du projet – une réussite en elle-même. Hugo Jacquit a posé les fondations du backend et du pipeline Spark ; Daniel Kouacou a poussé le projet au-delà de son périmètre initial en y intégrant un volet complet de machine learning ; Emy Regna a conçu et mis en place l'ensemble de la chaîne d'observabilité du projet.

Enfin, je remercie la communauté open-source (FastAPI, Apache Spark, PostgreSQL, Next.js, MLflow) dont la documentation et les forums ont été des ressources précieuses pour surmonter les obstacles techniques rencontrés.

Table des matières

Remerciements	1
1 Introduction	3
Accroche	3
Contexte et enjeux	3
Définition des termes clés	3
Présentation du projet FanFlight	5
Environnement technique et architecture	6
Annonce du plan	7
2 Compétences du référentiel RNCP38919 couvertes par le projet	8
2.1 BC01 – Concevoir un projet d’architecture technique de gestion de données	8
2.1.1 Identifier les besoins en architecture de gestion de données	8
2.1.2 Élaborer et exercer un système de veille technologique et réglementaire	8
2.1.3 Exploiter la veille au sein de l’équipe	8
2.1.4 Définir le périmètre du projet	9
2.1.5 Émettre des recommandations	9
2.2 BC02 – Élaborer une architecture technique de gestion de données	9
2.2.1 Collecter les données structurées et non structurées	9
2.2.2 Élaborer des solutions de stockage	9
2.2.3 Concevoir les procédures d’extraction, de transformation et de stockage	9
2.2.4 Transformer les données en un format approprié	10
2.2.5 Analyser les données	10
2.2.6 Automatiser les circuits de collecte, de traitement et de stockage	10
2.2.7 Développer un algorithme d’intelligence artificielle	10
2.3 BC03 – Déployer une solution d’analyse de données massives intégrant l’intelligence artificielle	10
2.3.1 Concevoir une interface de programmation	10
2.3.2 Conteneuriser les composants de l’architecture	10
2.3.3 Déployer le modèle dans un environnement de production	11
2.3.4 Orchestrer les services de la solution	11
2.3.5 Contrôler la mise en production de la solution	11
2.3.6 Automatiser le déploiement de nouvelles versions et son monitoring	11
2.4 BC04 – Piloter un projet d’architecture technique de gestion de données	11
2.4.1 Définir la structure organisationnelle du projet	11
2.4.2 Encadrer le développement du projet	12
2.4.3 Gérer le budget du projet	12
2.4.4 Communiquer l’avancement et les résultats du projet	12
2.4.5 Évaluer la performance du projet	12
2.4.6 Former les utilisateurs finaux de la solution	12
2.5 Synthèse de la couverture des compétences RNCP38919	13

3	Cahier des charges du projet FanFlight	14
3.1	Contexte et genèse du projet	14
3.2	Objectifs du projet	14
3.3	Contraintes du projet	14
3.4	Livrables attendus	15
4	Description de la démarche et des outils utilisés	16
4.1	Organisation du projet et répartition des rôles	16
4.2	Démarche technique : de l'amorçage à la mise en production	16
4.3	Outils de collaboration et méthodes de travail	17
4.4	Synthèse de la démarche	17
5	Réalisations du candidat	18
5.1	Modélisation des données : un schéma en étoile hybride	18
5.1.1	Présentation et objectif	18
5.1.2	Choix techniques	18
5.2	Pipeline ETL événementiel avec Apache Spark	19
5.2.1	Présentation et objectif	19
5.2.2	Transformation : inférence de schéma et clés de substitution	20
5.2.3	Chargement : staging puis fusion idempotente	20
5.3	API FastAPI : orchestration « cache d'abord, appel externe en repli »	21
5.3.1	Présentation et objectif	21
5.3.2	Choix techniques et limites	21
5.4	Frontend Next.js : un parcours guidé en trois étapes	21
5.4.1	Présentation et objectif	21
5.4.2	Composants principaux	22
5.5	Conteneurisation et intégration/déploiement continu	23
5.5.1	Orchestration locale	23
5.5.2	Intégration continue : construction et publication des images	23
5.5.3	Déploiement continu : SSH, Docker Compose et Watchtower	23
5.6	Observabilité de l'infrastructure : Prometheus et Grafana	24
5.6.1	Présentation et objectif	24
5.6.2	Une chaîne de collecte à plusieurs niveaux	24
5.6.3	Restitution : un tableau de bord Grafana provisionné	25
5.6.4	Intégration au déploiement	25
5.7	Extension Machine Learning : prédiction de retard de vol	25
5.7.1	Présentation et objectif	25
5.7.2	Un problème de cohérence des données résolu par une source alternative	25
5.7.3	Un contrat de features partagé entre entraînement et inférence	26
5.7.4	Entraînement, suivi et mise en service via MLflow	26
5.7.5	Résultats et limites assumées	26
5.7.6	Monitoring de dérive	26
6	Description d'une situation de travail ayant nécessité une recherche	27
6.1	Contexte de la problématique	27
6.2	Démarche de recherche	27
6.3	Implémentation de la solution	27
6.4	Itération ultérieure : un registre explicite des trajets déjà interrogés	28

7 Conclusion et ouverture	29
7.1 Synthèse du projet et réponse aux problématiques	29
7.2 Difficultés rencontrées et solutions apportées	29
7.3 Principaux bénéfices tirés du projet	30
7.4 Préconisations pour l'évolution du projet	30
7.5 Ouverture	31
Annexes	32
Annexe A : Documentation officielle	32
Annexe B : Sources de données externes	33
Annexe C : Blogs techniques et veille technologique	33
Annexe D : Communautés et forums	33
Annexe E : Outils et utilitaires	34
Annexe F : Certifications et formations	34
Annexe G : Ressources complémentaires	34

1. Introduction

Accroche

En 2026, la Coupe du Monde de football réunit pour la première fois 48 équipes et un nombre record de 104 matchs, répartis dans 16 villes hôtes à travers trois pays – les États-Unis, le Canada et le Mexique. Pour des millions de supporters, suivre son équipe favorite ne se limite plus à un aller-retour vers un stade voisin : cela implique de recouper un calendrier sportif dense avec une offre aérienne éclatée entre des dizaines de villes de départ et d'arrivée possibles, sur un continent entier. Or, aucun outil ne relie nativement ces deux mondes : le calendrier des matchs d'un côté, la disponibilité et le prix des vols de l'autre.

Contexte et enjeux

Planifier un déplacement autour d'un match de Coupe du Monde suppose aujourd'hui de jongler entre plusieurs sources : le calendrier officiel de la compétition, des moteurs de recherche de vols génériques, et une vérification manuelle de la cohérence des dates. Cette fragmentation de l'information ralentit la décision et complique l'expérience du supporter, en particulier lorsqu'il doit comparer plusieurs villes de départ ou plusieurs matchs.

Du point de vue technique, connecter ces deux mondes pose un problème d'ingénierie de données classique mais exigeant : il faut collecter une donnée sportive stable (calendrier, équipes, villes hôtes) et une donnée de vol volatile et coûteuse à interroger (les fournisseurs de données de vol facturent chaque requête), les stocker dans un modèle cohérent, les transformer de façon fiable et industrialisable, puis les exposer à une interface utilisateur simple – tout en maîtrisant un budget d'appels API et une infrastructure d'hébergement limités.

C'est ce double enjeu – rendre accessible une information aujourd'hui fragmentée, tout en construisant un pipeline de données robuste et économiquement soutenable – qui a motivé le choix du projet FanFlight comme projet fil rouge de notre cursus Data Engineer.

Définition des termes clés

API (Application Programming Interface)

Interface permettant à deux applications de communiquer. FastAPI expose une API REST consommée par le frontend Next.js de FanFlight.

BTS (Bureau of Transportation Statistics)

Agence fédérale américaine publiant des statistiques de ponctualité des vols domestiques. Utilisée dans l'extension machine learning de FanFlight comme source d'entraînement réelle.

CI/CD (Continuous Integration / Continuous Deployment)

Pratique consistant à automatiser la construction, la vérification et le déploiement du code. GitHub Actions orchestre ce cycle pour FanFlight.

Conteneur (Docker)

Unité logicielle packageant du code et ses dépendances pour une exécution reproductible. Chaque composant de FanFlight (frontend, API, Spark, base de données) est conteneurisé.

Data Warehouse / Schéma en étoile

Modélisation de données organisée autour de tables de faits (`FACT_FLIGHT`, `FACT_MATCHS`) reliées à des tables de dimensions (`DIM_CITY`, `DIM_AIRPORT`, `DIM_TEAM`). Modèle retenu pour la base PostgreSQL de FanFlight.

Docker Hub

Registre public d'images Docker. Les images de FanFlight y sont publiées sous le namespace `tezzosyris666`.

Drift (dérive de données)

Évolution statistique des données en production par rapport aux données d'entraînement d'un modèle. Surveillée dans FanFlight via des tests PSI et Kolmogorov-Smirnov.

ETL (Extract, Transform, Load)

Processus d'extraction, de transformation et de chargement de données. Implémenté par le pipeline Spark de FanFlight, qui transforme les réponses brutes de l'API de vols en enregistrements exploitables.

Event-driven architecture (architecture événementielle)

Architecture où un traitement est déclenché par un événement plutôt que par un ordonnancement périodique. Le pipeline Spark de FanFlight est déclenché par une notification PostgreSQL, pas par une tâche planifiée (cron).

FastAPI

Framework Python pour construire des API REST avec validation automatique des données et documentation OpenAPI générée. Framework backend de FanFlight.

GitHub Actions

Plateforme d'intégration et de déploiement continu intégrée à GitHub. Orchestre la construction des images Docker et le déploiement de FanFlight.

Grafana

Plateforme open-source de visualisation de métriques sous forme de tableaux de bord. Utilisée par FanFlight pour restituer les métriques collectées par Prometheus.

IATA

Code à trois lettres identifiant un aéroport (ex. `JFK`, `LAX`). Utilisé comme clé de correspondance entre villes et aéroports dans FanFlight.

JSONB

Type de colonne PostgreSQL stockant du JSON de façon binaire et indexable. Utilisé par FanFlight pour la table d'atterrissage `FLIGHTS_RAW`, qui conserve les réponses brutes de l'API de vols avant leur transformation.

LISTEN/NOTIFY

Mécanisme natif de PostgreSQL permettant à un processus de notifier des événements à des processus abonnés. Utilisé par FanFlight pour déclencher le pipeline Spark dès qu'une nouvelle donnée de vol brute est insérée.

MLflow

Plateforme open-source de suivi d'expériences et de registre de modèles de machine learning. Utilisée dans l'extension ML de FanFlight pour comparer et versionner les modèles de prédiction de retard.

MLOps

Ensemble de pratiques visant à industrialiser le cycle de vie des modèles de machine learning (entraînement, déploiement, monitoring). Illustré dans FanFlight par le registre de modèles MLflow et le monitoring de dérive.

Model Registry

Composant de MLflow qui versionne les modèles entraînés et gère leur promotion vers un statut « Production ». Utilisé pour le modèle `flight_delay_xgboost` de FanFlight.

Next.js

Framework React pour construire des applications web. Framework du frontend de FanFlight (version 16, App Router).

Open-Meteo

API météorologique gratuite et open-data. Utilisée dans FanFlight pour enrichir les données de vol avec des conditions météorologiques réelles ou prévisionnelles.

PostgreSQL

Système de gestion de base de données relationnelle open-source. Base de données centrale de FanFlight, épinglée en version 17.

Prometheus

Système open-source de collecte et de stockage de métriques temporelles, interrogé périodiquement (*scraping*) auprès de ses cibles. Collecteur central de la chaîne d'observabilité de FanFlight.

PySpark / Apache Spark

Framework de traitement de données distribué et son API Python. Moteur du pipeline ETL de FanFlight.

RGPD

Règlement Général sur la Protection des Données. FanFlight ne traite aucune donnée personnelle identifiante : la seule donnée utilisateur est une ville de départ sélectionnée dans une liste fermée.

RNCP

Répertoire National des Certifications Professionnelles. RNCP38919 « Data engineer » est le référentiel de certification du présent cursus.

SerpAPI

Service tiers exposant, entre autres, les résultats de recherche de vols de Google Flights sous forme d'API. Source de données de vol externe de FanFlight, facturée à la requête.

Surrogate key (clé de substitution)

Identifiant technique généré (ici par hachage MD5) plutôt qu'extrait directement de la donnée source. Utilisée par FanFlight (`journey_sk`, `flight_sk`) pour garantir l'idempotence des insertions.

Watchtower

Service qui surveille un registre Docker et met à jour automatiquement les conteneurs en production dès qu'une nouvelle image est disponible. Utilisé pour le déploiement continu de FanFlight sur l'instance EC2 de production.

XGBoost

Bibliothèque de gradient boosting sur arbres de décision, très utilisée sur données tabulaires. Modèle retenu pour la prédiction de retard de vol dans l'extension ML de FanFlight.

Présentation du projet FanFlight

FanFlight est une application web qui permet à un supporter de choisir sa ville de départ, de consulter les matchs de la Coupe du Monde 2026, puis de visualiser des propositions de vols cohérentes avec le calendrier du match sélectionné. Le parcours utilisateur se déroule en trois étapes guidées : sélection de la ville de départ, sélection du match, puis consultation des vols proposés. La version livrée couvre le calendrier de la phase de groupes de l'édition 2026 ; l'extension aux phases finales de la compétition est identifiée comme un axe d'évolution (cf. section 7.4).

Voici une capture d'écran de l'écran d'accueil de l'application FanFlight.

Le projet a été développé dans le cadre de notre formation Data Engineer chez Liora (certification DataScientest, RNCP38919) par une équipe de quatre personnes – Hugo Pradier, Hugo Jacquit, Daniel Kouacou et Emy Regna – répartie sur l'ensemble du territoire français et conciliant ce projet fil rouge avec nos alternances respectives. Contrairement à un scénario où l'équipe

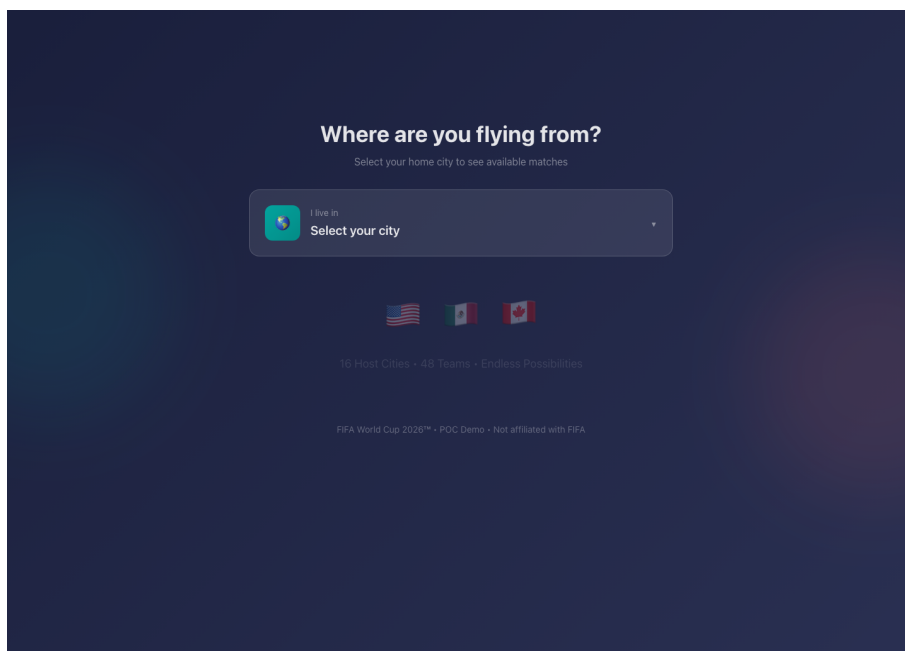
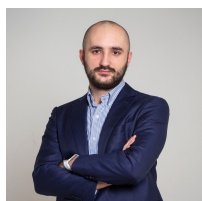
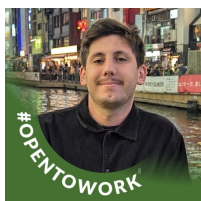


FIGURE 1.1 – Écran d'accueil de FanFlight – sélection de la ville de départ

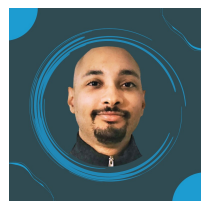
se serait délitée en cours de route, notre défi a été inverse : rester coordonnés à distance sur la durée du projet malgré ces contraintes (cf. section « Contraintes du projet »).

**Hugo Pradier**

Infrastructure, pipeline,
déploiement

**Hugo Jacquit**

Backend, pipeline Spark,
base de données

**Daniel Kouacou**

Extension machine
learning

**Emy Regna**

Observabilité
(Prometheus/Grafana)

FIGURE 1.2 – L'équipe du projet FanFlight

Environnement technique et architecture

FanFlight repose sur une architecture conteneurisée orchestrée par Docker Compose, structurée autour des technologies suivantes :

- **Frontend** : Next.js 16 (App Router) / React 19 / Tailwind CSS 4 / framer-motion
- **Backend** : FastAPI (Python), API REST documentée automatiquement (OpenAPI)
- **Base de données** : PostgreSQL 17, schéma en étoile hybride relationnel / JSONB
- **Pipeline de données** : Apache Spark (PySpark), déclenché par événements PostgreSQL (LISTEN/NOTIFY)
- **Source de données externe** : SerpAPI (moteur `google_flights`)
- **Extension Machine Learning** : XGBoost / PyTorch, suivi et registre via MLflow
- **Observabilité** : Prometheus, Grafana, Blackbox Exporter, node-exporter, cAdvisor, postgres-exporter

- **Conteneurisation** : Docker, images publiées sur Docker Hub (namespace `tezzosyris666`)
- **CI/CD** : GitHub Actions (build, push, déploiement SSH)
- **Déploiement continu** : Watchtower (mise à jour automatique des conteneurs en production)
- **Hébergement de production** : instance AWS EC2, fournie par notre école en cours de projet

La figure 1.3 synthétise les flux entre ces composants.

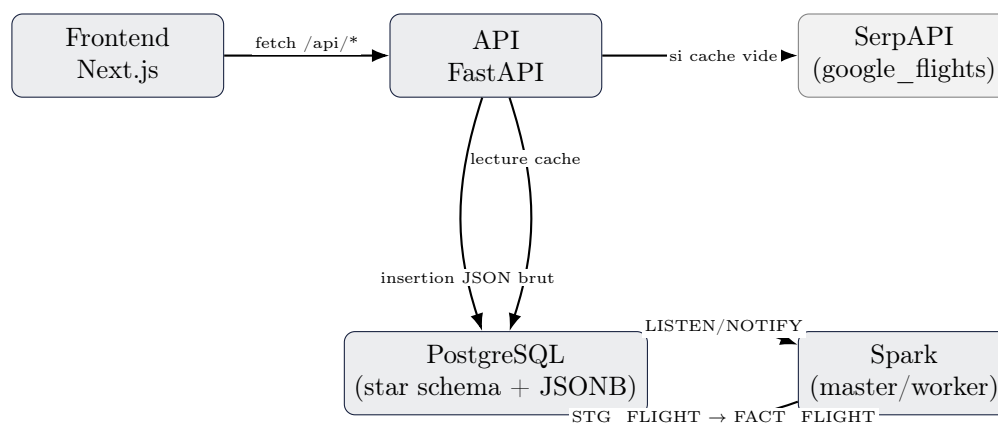


FIGURE 1.3 – Architecture applicative et flux de données de FanFlight

Annnonce du plan

Ce dossier présente d'abord les compétences du référentiel RNCP38919 mobilisées par le projet FanFlight. Nous détaillerons ensuite le cahier des charges du projet, puis la démarche et les outils utilisés pour le mener à bien en équipe distribuée. La présentation de nos réalisations techniques concrètes sera suivie d'une analyse d'une problématique de recherche rencontrée en cours de projet – le contrôle du coût des appels à l'API externe de vols. Nous concluons par une synthèse des difficultés rencontrées, des bénéfices tirés du projet, et des perspectives d'évolution.

2. Compétences du référentiel RNCP38919 couvertes par le projet

Le titre « Data engineer » (RNCP38919, niveau 7, certificateur DataScientest) est structuré en quatre blocs de compétences (BC01 à BC04). Le projet FanFlight a été construit pour couvrir l'intégralité de ces blocs et en démontrer la maîtrise de manière concrète.

2.1 BC01 – Concevoir un projet d'architecture technique de gestion de données

2.1.1 Identifier les besoins en architecture de gestion de données

Le besoin a été formalisé à partir d'un constat simple : un supporter de la Coupe du Monde 2026 doit croiser manuellement deux sources d'information (calendrier des matchs et offres de vols) sans qu'aucun outil ne les relie. Nous avons analysé le flux de données nécessaire – données sportives stables (équipes, villes hôtes, calendrier) et données de vol volatiles et coûteuses à obtenir (SerpAPI) – pour valider l'opportunité et le périmètre technique du projet.

2.1.2 Élaborer et exercer un système de veille technologique et réglementaire

Notre veille technique et réglementaire a porté sur plusieurs axes structurants :

- comparaison de fournisseurs de données de vol (API dédiées vs scraping) et sélection de SerpAPI pour son accès à Google Flights ;
- évaluation d'une architecture de traitement par lot (cron) contre une architecture événementielle – retenue pour limiter la latence perçue côté utilisateur ;
- étude des mécanismes natifs PostgreSQL (`LISTEN/NOTIFY`) comme alternative légère à une file de messages (Kafka, RabbitMQ), jugée disproportionnée pour le volume et le budget du projet ;
- veille sur les pratiques de MLOps (MLflow, model registry, monitoring de dérive) pour l'extension machine learning ;
- veille réglementaire sur la protection des données (RGPD), pour vérifier qu'un traitement ne collectant aucune donnée personnelle identifiante restait hors du champ des obligations les plus contraignantes, et sur les recommandations relatives à l'explicabilité des modèles de scoring, qui ont motivé le recours à SHAP pour documenter les prédictions du modèle de retard (cf. section 5.7).

2.1.3 Exploiter la veille au sein de l'équipe

Les arbitrages techniques ont été partagés et discutés entre les quatre membres de l'équipe et avec notre mentor Dan Cohen, malgré l'absence de coprésence physique. Les décisions d'architecture (schéma en étoile, pipeline événementiel, stratégie de cache) ont fait l'objet de revues de code via des *pull requests* sur GitHub avant fusion sur la branche principale.

2.1.4 Définir le périmètre du projet

Le périmètre a été formalisé autour de : (1) un socle de données de référence figées pour l'édition 2026 (équipes, villes hôtes, calendrier des matchs de phase de groupes – l'extension aux phases finales étant un axe d'évolution identifié, cf. section 7.4), (2) une couche dynamique de vols alimentée à la demande, (3) une interface de consultation. Les enjeux réglementaires ont été jugés limités : FanFlight ne collecte aucune donnée personnelle identifiante, la seule saisie utilisateur étant une ville de départ choisie dans une liste fermée. L'enjeu principal identifié était économique : le coût par appel de l'API externe de vols imposait de concevoir le système pour minimiser les requêtes redondantes dès la conception (cf. chapitre 6). Ce même arbitrage a aussi guidé un choix à faible empreinte : privilégier les mécanismes déjà natifs de PostgreSQL (`LISTEN /NOTIFY`) plutôt que d'ajouter un service dédié supplémentaire (file de messages, cache Redis) limite le nombre de conteneurs actifs en permanence, et donc la consommation de ressources de l'infrastructure – un objectif de sobriété numérique qui rejoignait naturellement la contrainte budgétaire.

2.1.5 Émettre des recommandations

Les choix d'architecture ont été présentés et défendus au sein de l'équipe puis auprès de notre mentor : modèle de données en étoile plutôt que documentaire, pipeline événementiel plutôt que batch, cache applicatif porté par PostgreSQL plutôt que par une brique dédiée (Redis) pour ne pas alourdir l'infrastructure à opérer par une petite équipe.

2.2 BC02 – Élaborer une architecture technique de gestion de données

2.2.1 Collecter les données structurées et non structurées

FanFlight combine deux natures de données : des données structurées codées en dur pour l'édition 2026 (équipes, villes hôtes et leurs aéroports, calendrier des matchs, dans `load_static_data.py`) et des données semi-structurées collectées dynamiquement (réponses JSON de SerpAPI), stockées telles quelles avant transformation. Un référentiel externe (`airport-codes.csv`) est également mobilisé pour enrichir les aéroports inconnus au fil de l'utilisation.

2.2.2 Élaborer des solutions de stockage

Le stockage repose sur PostgreSQL, avec un modèle hybride :

- un schéma en étoile relationnel classique – tables de dimensions `DIM_CITY`, `DIM_AIRPORT`, `DIM_TEAM` et tables de faits `FACT_MATCHS`, `FACT_FLIGHT` ;
- une table d'atterrissage semi-structurée `FLIGHTS_RAW` en colonne JSONB, qui conserve la réponse brute de l'API externe avant sa transformation, avec un indicateur `processed` servant de suivi (*watermark*) du pipeline.

2.2.3 Concevoir les procédures d'extraction, de transformation et de stockage

Le pipeline suit un schéma classique *staging puis fusion* : les données brutes sont extraites de la table d'atterrissage `FLIGHTS_RAW`, transformées par un job Spark dans une table de *staging* (`STG_FLIGHT`), puis fusionnées de façon idempotente dans la table de faits finale (`FACT_FLIGHT`) via une clause `ON CONFLICT ... DO UPDATE`. Ce détail est développé au chapitre 5.

2.2.4 Transformer les données en un format approprié

La transformation Spark infère dynamiquement le schéma JSON de chaque réponse SerpAPI (`schema_of_json`), sépare les propositions « meilleurs vols » et « autres vols », éclate chaque itinéraire en segments de vol (`posexplode`), calcule des clés de substitution par hachage MD5 pour garantir l'unicité et l'idempotence, gère de façon défensive les champs optionnels (une correspondance – *layover* – n'existe pas pour un vol direct), et déduplique les segments avant chargement.

2.2.5 Analyser les données

La qualité et la disponibilité des données sont évaluées à chaque requête : la fonction `check_flight_db` vérifie si un trajet aller valide existe déjà en base – un vol reliant la ville de départ de l'utilisateur à la ville du match, dont l'arrivée se situe entre 3 jours avant le match et, au plus tard, 5 heures avant son coup d'envoi – avant de déclencher un nouvel appel à l'API externe. Les résultats sont ensuite restitués à l'utilisateur final via l'interface de vols. À une échelle plus large, l'évaluation de la qualité des données a également été formalisée dans l'extension machine learning du projet, via des tests statistiques de dérive (indice PSI, test de Kolmogorov-Smirnov, cf. section 5.7) dont les résultats ont vocation à être partagés avec l'équipe et notre mentor lors des points d'avancement.

2.2.6 Automatiser les circuits de collecte, de traitement et de stockage

L'ensemble du circuit est automatisé de bout en bout : une notification PostgreSQL déclenche automatiquement le job Spark, qui déclenche à son tour la fusion vers la table de faits et la mise à jour du statut de traitement – sans intervention manuelle. Le script `entrypoint.sh` du conteneur API rejoue, à chaque démarrage, l'initialisation du schéma et le chargement des données de référence – deux étapes rendues idempotentes par des clauses `ON CONFLICT`. Il exécute aussi systématiquement `replay_job.py`, qui réinjecte les réponses SerpAPI déjà sauvegardées sur disque : cette dernière étape n'est, elle, pas idempotente et déclenche un nouveau job Spark à chaque redémarrage du conteneur (cf. chapitre 6 et section 7.4).

2.2.7 Développer un algorithme d'intelligence artificielle

Un algorithme de classification binaire (probabilité de retard de vol) a été développé par Daniel Kouacou sur une branche dédiée du projet, combinant des données réelles de ponctualité aérienne américaine (BTS) et des données météorologiques (Open-Meteo), avec deux modèles comparés (XGBoost et un réseau de neurones PyTorch). Ce travail est détaillé en section 5.7.

2.3 BC03 – Déployer une solution d'analyse de données massives intégrant l'intelligence artificielle

2.3.1 Concevoir une interface de programmation

L'API FastAPI constitue l'interface unique entre le frontend, la base de données et – dans l'extension machine learning – le modèle de prédiction. Elle expose des routes REST simples (`/api/matches`, `/api/teams`, `/api/cities`, `/api/flights/{match_id}`) et bénéficie d'une documentation OpenAPI générée automatiquement par FastAPI.

2.3.2 Conteneuriser les composants de l'architecture

Chaque composant (frontend, API, Spark) dispose de son propre `Dockerfile` et de sa propre image, publiée sur Docker Hub. Ces images n'installent que les paquets système strictement nécessaires – par exemple `gcc/libpq-dev` dans l'image Spark, requis pour compiler `psycopg2`

(le pilote Python utilisé par `spark_listener.py` et `stg_to_fact.py`), à ne pas confondre avec le pilote JDBC PostgreSQL utilisé par Spark lui-même, ajouté comme binaire préconstruit et ne nécessitant aucune compilation. L'accès aux registres et à l'infrastructure de production est protégé par une gestion de secrets : les identifiants Docker Hub et la clé SSH de déploiement sont stockés comme secrets GitHub Actions, jamais en clair dans le code versionné. Seule l'image frontend s'exécute toutefois avec un utilisateur non privilégié dédié ; les images API et Spark tournent encore avec un utilisateur root, ce qui constitue une piste de durcissement identifiée pour la suite du projet (cf. section 7.4).

2.3.3 Déployer le modèle dans un environnement de production

Dans l'extension machine learning, le modèle est versionné et promu au statut « Production » via le registre de modèles MLflow, puis chargé par l'API à son démarrage (`hook @app.on_event("startup")`) pour scorer chaque vol renvoyé par l'API en temps réel.

2.3.4 Orchestrer les services de la solution

Docker Compose orchestre l'ensemble des services : base de données, cluster Spark (*master* et *worker*), API, frontend, et – pour l'extension ML – serveur MLflow. Les dépendances de démarrage sont explicitées via des conditions `depends_on` (par exemple l'API n'accepte des requêtes qu'une fois la base de données déclarée saine par son *healthcheck*).

2.3.5 Contrôler la mise en production de la solution

Un fichier `docker-compose.local.yml` permet de rejouer, en local, une configuration proche de la production (images pré-construites, mêmes variables d'environnement) avant tout déploiement réel. Le pipeline de déploiement inclut par ailleurs une étape de diagnostic automatique (relevé des journaux des conteneurs `postgres` et `backend_api`) en cas d'échec du démarrage des services sur l'instance de production.

2.3.6 Automatiser le déploiement de nouvelles versions et son monitoring

Le déploiement continu repose sur GitHub Actions (construction et publication des images à chaque *push* sur la branche principale) et sur Watchtower, qui surveille Docker Hub toutes les 60 secondes et met à jour automatiquement les conteneurs de production dès qu'une nouvelle image `latest` est disponible. Le monitoring de l'infrastructure elle-même est couvert par la chaîne d'observabilité développée par Emy Regna (Prometheus, Grafana, cf. section 5.6), et le monitoring de l'évolution des données par un script de détection de dérive pour l'extension ML (`drift_monitor.py`, tests PSI et Kolmogorov–Smirnov).

2.4 BC04 – Piloter un projet d'architecture technique de gestion de données

2.4.1 Définir la structure organisationnelle du projet

Le projet a été structuré en quatre flux de travail répartis selon les affinités de chacun : Hugo Pradier sur l'infrastructure, l'intégration du pipeline de données et le déploiement ; Hugo Jacquit sur l'amorçage du backend, du pipeline Spark et de la base de données ; Daniel Kouacou sur l'extension machine learning ; Emy Regna sur la chaîne d'observabilité de l'infrastructure (Prometheus, Grafana). Cette organisation a dû composer avec une contrainte forte, détaillée au chapitre 3 : une équipe répartie sur toute la France, chacun en alternance en entreprise en parallèle du projet.

2.4.2 Encadrer le développement du projet

Le développement a suivi une démarche itérative inspirée des pratiques agiles, adaptée à une équipe distribuée ne pouvant pas se caler sur des rituels synchrones quotidiens : le travail a été découpé en lots livrables et intégrables indépendamment (amorçage, socle backend/Spark/API, fiabilisation de la base, intégration ETL/frontend, durcissement du déploiement, extension machine learning – cf. chapitre 5 et section « Démarche technique »), chacun validé par une revue systématique de code avant fusion sur la branche principale – trois de ces lots ont ainsi fait l’objet d’une fusion formelle par *pull request*. Un accompagnement régulier de notre mentor Dan Cohen a complété ce cadre, avec un rôle proche de celui d’un *product owner* : validation des choix d’architecture et priorisation des lots suivants. Le travail à distance a par ailleurs nécessité une discipline de communication asynchrone plus poussée qu’un travail en présentiel n’en aurait exigé.

2.4.3 Gérer le budget du projet

Deux postes de coût ont structuré les arbitrages du projet : le coût par requête de l’API externe de vols (SerpAPI), qui a directement orienté la conception du pipeline vers une architecture minimisant les appels redondants (cf. chapitre 6), et le coût d’hébergement de la production, initialement incertain, résolu lorsque notre école (Liora) a mis à notre disposition une instance AWS EC2 en fin de projet. Le suivi du premier poste a porté sur la conception même des requêtes envoyées à SerpAPI : chaque recherche interroge toutes les combinaisons d’aéroports possibles entre la ville de départ et la ville d’arrivée, sur les 3 jours précédant le match – un facteur multiplicatif qu’il fallait garder à l’esprit pour ne déclencher ce coût qu’au moment d’un véritable cache manqué, jamais de façon spéculative.

2.4.4 Communiquer l’avancement et les résultats du projet

L’avancement a été communiqué régulièrement à notre mentor et synthétisé au fil de l’historique Git (structuration initiale, intégration backend/Spark/API, correctifs de base de données, intégration ETL/frontend, durcissement du déploiement), permettant un suivi transparent de la progression malgré le travail à distance.

2.4.5 Évaluer la performance du projet

La performance du pipeline a été évaluée sur des critères concrets : taux de réussite du cache de vols (proportion de requêtes servies directement depuis PostgreSQL sans nouvel appel à SerpAPI), respect du délai maximal de 90 secondes toléré pour un cycle complet d’enrichissement asynchrone, et – pour le volet machine learning – l’aire sous la courbe ROC (AUC) des modèles de prédiction de retard (0,728 pour XGBoost).

2.4.6 Former les utilisateurs finaux de la solution

Le plan d’accompagnement conçu pour les utilisateurs finaux de FanFlight repose sur le parcours lui-même : un assistant en trois étapes (ville, match, vols) où chaque écran ne présente que l’action suivante possible, pensé pour ne nécessiter aucune documentation externe. Ce parcours a été affiné au fil des points d’avancement avec notre mentor, qui a testé l’interface comme le ferait un utilisateur final et dont les retours ont permis d’ajuster son déroulé. Pour les utilisateurs techniques (contributeurs futurs, mainteneurs), la documentation OpenAPI générée automatiquement par FastAPI et les fichiers du dossier `documentation/` du dépôt jouent ce même rôle d’accompagnement.

2.5 Synthèse de la couverture des compétences RNCP38919

Bloc	Compétence	Mise en œuvre dans FanFlight
BC01 – Concevoir	Identifier les besoins, veille, périmètre	Analyse du besoin supporter/vols, veille SerpAPI/architecture événementielle, périmètre formalisé en équipe
	Émettre des recommandations	Choix défendus en équipe et avec le mentor (schéma en étoile, pipeline événementiel)
BC02 – Élaborer	Collecter les données	Données statiques (équipes, villes, calendrier) + données SerpAPI dynamiques
	Élaborer le stockage	PostgreSQL, schéma en étoile + table d’atterrissage JSONB <code>FLIGHTS_RAW</code>
	Concevoir l’ETL	Pipeline Spark événementiel (staging → fusion upsert)
	Transformer et analyser	Inférence de schéma, clés de substitution MD5, requêtes de disponibilité
	Automatiser les circuits	<code>LISTEN/NOTIFY</code> + <code>entrypoint.sh</code> idempotent
BC03 – Déployer	Développer un algorithme d’IA	Modèle XGBoost / PyTorch de prédiction de retard (BTS + météo)
	Interface de programmation	API REST FastAPI, documentation OpenAPI
	Conteneuriser	Docker par composant, images publiées sur Docker Hub
	Déployer le modèle	Registre MLflow, chargement au démarrage de l’API
	Orchestrer les services	Docker Compose (DB, Spark, API, frontend, MLflow)
	Contrôler la mise en production	Compose « prod-like » local avant déploiement réel
BC04 – Piloter	Automatiser déploiement/monitoring	GitHub Actions + Watchtower + Prometheus/Grafana + <code>drift_monitor.py</code>
	Structure organisationnelle	4 rôles définis, équipe distribuée + alternances
	Gérer le budget	Architecture pensée pour maîtriser le coût SerpAPI; EC2 fourni par l’école
	Évaluer la performance	Taux de cache, délai de pipeline, AUC du modèle ML

3. Cahier des charges du projet FanFlight

3.1 Contexte et genèse du projet

L'idée de FanFlight est née du constat que la Coupe du Monde 2026 – première édition à 48 équipes, disputée dans 16 villes hôtes de trois pays différents – allait démultiplier le besoin, pour les supporters, de déplacements complexes à planifier. Dans le cadre de notre formation Data Engineer chez Liora (certification DataScientest, RNCP38919), ce projet a été retenu comme projet fil rouge par une équipe de quatre personnes : Hugo Pradier, Hugo Jacquit, Daniel Kouacou et Emy Regna.

À la différence d'un projet d'équipe classique en présentiel, notre contexte de travail a été marqué dès le départ par une contrainte structurelle géographique et calendaire, détaillée ci-après (section « Contraintes du projet »).

3.2 Objectifs du projet

Objectifs fonctionnels :

- permettre à un utilisateur de sélectionner une ville de départ et de consulter les matchs de la Coupe du Monde 2026 ;
- proposer, pour un match donné, des vols cohérents avec le calendrier (arrivée dans la ville du match jusqu'à 3 jours avant sa date, et au plus tard 5 heures avant son coup d'envoi) ;
- restituer ces informations dans une interface simple et guidée.

Objectifs techniques et data engineering (prioritaires pour la certification) :

- concevoir un modèle de données combinant un référentiel stable et une donnée dynamique coûteuse à obtenir ;
- construire un pipeline ETL automatisé et idempotent avec Apache Spark ;
- exposer les données via une API REST documentée ;
- conteneuriser l'ensemble de la solution et automatiser son déploiement (CI/CD) ;
- maîtriser le coût des appels à l'API externe de vols dès la conception de l'architecture ;
- mettre en place une chaîne d'observabilité de l'infrastructure (supervision système, applicative et réseau) ;
- explorer l'intégration d'un algorithme de machine learning dans la chaîne de données (prédiction de retard de vol).

3.3 Contraintes du projet

Contraintes organisationnelles :

- équipe de quatre personnes réparties sur l'ensemble du territoire français, sans coprésence physique possible ;
- chaque membre de l'équipe conciliait le projet fil rouge avec une alternance en entreprise, réduisant le temps disponible et rendant la synchronisation des plannings délicate ;

- durée contrainte du cursus bootcamp.

Contraintes techniques et budgétaires :

- l'API externe de vols (SerpAPI) facture chaque requête, ce qui interdisait une architecture naïve interrogeant l'API à chaque consultation utilisateur ;
- aucun budget d'hébergement de production n'était garanti au démarrage du projet ; une instance AWS EC2 n'a été mise à disposition par notre école (Liora) qu'en fin de projet, ce qui a orienté le développement initial vers un environnement entièrement local (Docker Compose) validé avant tout déploiement réel.

3.4 Livrables attendus

- une application FanFlight fonctionnelle et déployée en production (frontend, API, base de données, pipeline Spark) ;
- un pipeline de données automatisé, événementiel et idempotent ;
- une chaîne CI/CD fonctionnelle (GitHub Actions, Docker Hub, déploiement automatisé, mise à jour continue via Watchtower) ;
- une documentation technique du déploiement et de l'exploitation ;
- une chaîne d'observabilité de l'infrastructure (Prometheus, Grafana) restituée dans un tableau de bord unique ;
- une extension machine learning démontrant l'intégration d'un algorithme d'intelligence artificielle dans la chaîne de données.

4. Description de la démarche et des outils utilisés

4.1 Organisation du projet et répartition des rôles

Le projet a été réparti selon quatre axes de responsabilité, en cohérence avec les affinités techniques de chacun :

- **Hugo Jacquit** a posé les fondations du projet : structuration initiale du backend, du pipeline Spark et de l'API (première intégration `add base+spark+api`), puis correction de la logique d'initialisation de la base de données et de problèmes d'encodage de caractères.
- **Hugo Pradier** a pris en charge l'intégration de bout en bout du pipeline ETL avec le frontend, la correction de la communication entre l'API et le frontend, puis l'ensemble de la chaîne de déploiement : configuration SSH/production, workflow GitHub Actions, documentation de déploiement, fiabilisation de la base (épinglage de version PostgreSQL) et rédaction du présent dossier.
- **Daniel Kouacou** a conduit, sur une branche dédiée, l'extension machine learning du projet : collecte de données réelles de ponctualité aérienne (BTS), enrichissement météorologique, feature engineering, entraînement et suivi de modèles via MLflow, et monitoring de dérive des données.
- **Emy Regna** a conçu et mis en place l'ensemble de la chaîne d'observabilité du projet : stack Prometheus/Grafana, exporters système et applicatifs (node-exporter, cAdvisor, postgres-exporter), health-checks HTTP via Blackbox Exporter, et le dashboard de supervision (cf. section 5.6).

4.2 Démarche technique : de l'amorçage à la mise en production

L'historique du projet reflète une progression itérative, cohérente avec les contraintes de calendrier de l'équipe :

1. **Amorçage** – mise en place de la structure du dépôt et des conventions de projet.
2. **Intégration du socle technique** (*pull request #1*) – première version fonctionnelle de la base de données, du pipeline Spark et de l'API, fusionnée depuis la branche de travail de Hugo Jacquit.
3. **Fiabilisation de la base de données** (*pull request #2*) – correction de la logique d'initialisation du schéma et de problèmes d'encodage.
4. **Intégration ETL/frontend** (*pull request #3*) – raccordement du pipeline de transformation Spark au frontend et correction de la communication entre l'API et le frontend.
5. **Durcissement du déploiement** – mise en place de la connexion SSH sécurisée vers l'instance de production, du workflow GitHub Actions complet, de la documentation de déploiement, suppression du cache de build devenu source d'incohérences, et épinglage explicite de la version majeure de PostgreSQL pour éviter une rupture de compatibilité de volume en production.
6. **Extension machine learning** – développement, en parallèle et sur une branche dédiée, d'un module de prédiction de retard de vol.

Cette progression illustre une caractéristique propre à un projet mené par une équipe distribuée et contrainte par ailleurs : les intégrations se sont faites par lots, validées par revue de code, plutôt que par un travail synchrone continu.

4.3 Outils de collaboration et méthodes de travail

Face à ces contraintes (cf. chapitre 3), nous avons privilégié des outils et pratiques de collaboration asynchrone :

- **Git / GitHub** : développement sur des branches de fonctionnalité dédiées (une par contributeur ou par lot de travail), intégrées à la branche principale par *pull request* revue avant fusion – les étapes 2, 3 et 4 de la démarche technique ci-dessus (socle technique, fiabilisation de la base, intégration ETL/frontend) correspondent chacune à l’une des trois *pull requests* qui ont structuré le projet ;
- **Docker Compose** : environnement de développement reproductible, permettant à chaque membre de l’équipe de retrouver un environnement identique malgré l’absence de poste de travail partagé ;
- **Suivi mentoré** : accompagnement régulier de Dan Cohen pour valider les choix d’architecture et déboguer les blocages techniques ;
- **Communication asynchrone** : échanges écrits pour la majorité des décisions, afin de s’adapter aux emplois du temps disjoints imposés par nos alternances.

4.4 Synthèse de la démarche

Le projet FanFlight illustre une progression méthodique typique d’un projet data engineering mené en équipe distribuée : un amorçage rapide, une intégration du socle technique par lots validés en revue de code, puis un durcissement progressif du déploiement jusqu’à son automatisation complète. La contrainte organisationnelle – une équipe dispersée géographiquement et disponible par créneaux – a, en retour, renforcé la rigueur de nos pratiques de revue de code et de documentation, plutôt que de constituer un obstacle bloquant.

5. Réalisations du candidat

Cette section présente les composants techniques les plus significatifs du projet FanFlight, en justifiant les choix effectués.

5.1 Modélisation des données : un schéma en étoile hybride

5.1.1 Présentation et objectif

Le script `API/init_database/init_db.py` (environ 115 lignes) définit le schéma de la base PostgreSQL. Il combine un schéma en étoile relationnel classique pour les données de référence et une table d’atterrissage semi-structurée pour les données de vol brutes.

Listing 5.1 – Extrait du schéma – table de faits `FACT_FLIGHT` et table d’atterrissage `FLIGHTS_RAW`

```
CREATE TABLE IF NOT EXISTS FACT_FLIGHT (
    flight_sk VARCHAR(32) PRIMARY KEY,
    journey_sk VARCHAR(32),
    departure_airport_code VARCHAR(10),
    departure_airport_time TIMESTAMP,
    arrival_airport_code VARCHAR(10),
    arrival_airport_time TIMESTAMP,
    duration INTEGER,
    flight_number VARCHAR(12),
    layover_duration INTEGER,
    total_journey_duration INTEGER,
    price DECIMAL(10, 2),
    airline VARCHAR(100),
    airline_logo TEXT,
    type VARCHAR(20),
    is_best BOOLEAN,
    pos INTEGER,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS FLIGHTS_RAW (
    id SERIAL PRIMARY KEY,
    json_data JSONB,
    processed BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5.1.2 Choix techniques

- **Clé de substitution** `flight_sk` : identifiant technique calculé par hachage plutôt qu’auto-incrémenté, ce qui permet à la clause `ON CONFLICT (flight_sk) DO UPDATE` de rendre les rechargements du pipeline naturellement idempotents.

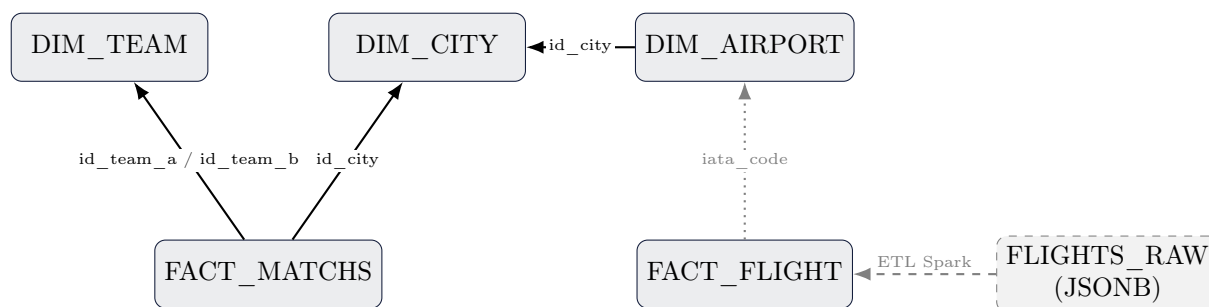


FIGURE 5.1 – Modèle de données FanFlight – clés étrangères déclarées (traits pleins), jointure applicative non contrainte en base entre `FACT_FLIGHT` et `DIM_AIRPORT` (pointillés), et alimentation de `FACT_FLIGHT` depuis la table d’atterrissage `FLIGHTS_RAW` par le pipeline ETL (tirets)

- **Regroupement par** `journey_sk` : chaque itinéraire (potentiellement multi-segments) partage une même clé de trajet, la colonne `pos` ordonnant les segments entre eux.
- **Table** `FLIGHTS_RAW` en JSONB : conserver la réponse brute de l’API externe permet de rejouer une transformation en cas d’évolution du pipeline, sans nouvel appel (payant) à l’API.
- **Chargement des données de référence** (`load_static_data.py`, environ 355 lignes) : équipes, villes hôtes, aéroports et calendrier des matches de l’édition 2026 sont insérés via des clauses `ON CONFLICT DO NOTHING / DO UPDATE`, rendant le script rejouable sans risque de doublon.
- **Jointure applicative** `FACT_FLIGHT` / `DIM_AIRPORT` : contrairement aux autres relations du schéma, le lien entre un vol et son aéroport n’est pas déclaré comme contrainte de clé étrangère en base – il n’est vérifié qu’au moment des requêtes SQL applicatives (`JOIN ... ON f.departure_airport_code = a.iata_code`). Ce choix évite un rejet en base si `enrich_airports` n’a pas encore inséré un aéroport rencontré pour la première fois, au prix d’une intégrité référentielle non garantie par le SGBD – une piste de durcissement identifiée (cf. section 7.4).

5.2 Pipeline ETL événementiel avec Apache Spark

5.2.1 Présentation et objectif

Plutôt qu’un pipeline ordonnancé (cron/Airflow), FanFlight utilise le mécanisme natif `LISTEN /NOTIFY` de PostgreSQL pour déclencher la transformation Spark dès qu’une nouvelle réponse de l’API de vols est insérée – une architecture événementielle légère, adaptée au volume et au budget d’infrastructure du projet.

Listing 5.2 – `spark_listener.py` – déclenchement événementiel du job Spark

```

cursor.execute("LISTEN new_flight;")

while True:
    try:
        select.select([conn], [], [])
        conn.poll()
        for notify in conn.notifies:
            job_id = notify.payload
            subprocess.Popen([
                "/opt/spark/bin/spark-submit",
                "--master", "spark://spark-master:7077",
                "/opt/spark/scripts/process_flight.py",

```

```

        job_id
    })
    conn.notifies.clear()
except Exception as e:
    print(f"Erreur : {e}")

```

5.2.2 Transformation : inférence de schéma et clés de substitution

Le script `process_flight.py` (environ 175 lignes) lit la ligne `FLIGHTS_RAW` correspondant au job via JDBC, infère dynamiquement le schéma JSON de la réponse SerpAPI, puis éclate les itinéraires « meilleurs vols » et « autres vols » en segments individuels :

Listing 5.3 – `process_flight.py` – calcul des clés de substitution par hachage

```

df_best_flights = df_best_flights.withColumn("journey_sk",
    md5(concat_ws('_',
        col("bf.flights.departure_airport.id"),
        col("bf.flights.arrival_airport.id"),
        col("bf.flights.airline"),
        col("bf.flights.flight_number"),
        to_date(col("bf.flights")[0]["departure_airport"]["time"]))))
    # date sans l'heure : la cle ne change pas si l'horaire est ajuste

df_detail_best_flight = df_detail_best_flight.withColumn("flight_sk",
    md5(concat_ws('_', col("journey_sk"), col("flight.flight_number"),
        col("flight.departure_airport.id"), col("pos")))))

```

Un point d'attention technique a été géré de façon défensive : les vols directs ne possèdent pas de champ `layovers` (correspondances) dans la réponse SerpAPI. Le code vérifie la présence de la colonne avant de la sélectionner, avec une valeur nulle typée en repli, pour éviter une erreur de schéma sur les vols sans escale.

5.2.3 Chargement : staging puis fusion idempotente

Après union des deux branches (meilleurs vols / autres vols) et déduplication sur `flight_sk`, les données sont écrites en mode `overwrite` dans une table de *staging* (`STG_FLIGHT`), puis fusionnées vers la table de faits par le script `stg_to_fact.py` (environ 45 lignes) :

Listing 5.4 – `stg_to_fact.py` – fusion idempotente staging vers fait

```

INSERT INTO fact_flight (flight_sk, journey_sk, departure_airport_code, ...)
SELECT flight_sk, journey_sk, departure_airport_code, ...
FROM stg_flight
ON CONFLICT (flight_sk)
DO UPDATE SET
    price = EXCLUDED.price,
    departure_airport_time = EXCLUDED.departure_airport_time,
    updated_at = CURRENT_TIMESTAMP;

UPDATE flights_raw SET processed = True WHERE id = %s;

```

Cette dernière instruction referme la boucle événementielle : c'est la mise à jour du drapeau `processed` qui permet à l'API, qui interroge ce champ en boucle, de savoir que le pipeline a terminé son travail.

5.3 API FastAPI : orchestration « cache d’abord, appel externe en repli »

5.3.1 Présentation et objectif

Le fichier `API/main.py` (environ 490 lignes) est le point d’entrée unique du frontend. Sa route la plus significative, `GET /api/flights/{match_id}`, matérialise directement la contrainte de coût du projet : elle ne déclenche un appel à l’API externe payante qu’en dernier recours.

Listing 5.5 – `main.py` – logique cache d’abord, appel externe en repli

```
flights = check_flight_db(cursor, departure_city, match_info['arrival_city'],
    match_date_exact)

if flights:
    return {"meta": {...}, "flights": flights}

if not flights:
    job_ids = get_flights_api(cursor, conn, departure_city,
        match_info['arrival_city'], match_date_exact)

    if not job_ids:
        return {"meta": {...}, "flights": []}

    wait_to_spark(cursor, job_ids)
    enrich_airports(cursor, conn)
    flights = check_flight_db(cursor, departure_city,
        match_info['arrival_city'], match_date_exact)
    return {"meta": {...}, "flights": flights}
```

`check_flight_db` construit une requête SQL à base de CTE qui recherche des itinéraires *aller simple* dont le premier segment part de la ville de l’utilisateur et dont le dernier segment arrive dans la ville du match, avec une contrainte temporelle métier sur ce seul trajet : arrivée au plus tôt 3 jours avant le match, et au plus tard 5 heures avant son coup d’envoi – la fonction ne modélise pas de vol retour. En cas d’absence de résultat, `get_flights_api` interroge SerpAPI pour chaque paire d’aéroports possible sur les 3 jours précédant le match, insère la réponse brute dans `FLIGHTS_RAW`, puis `wait_to_spark` attend – par interrogation périodique du drapeau `processed`, avec un délai maximal de 90 secondes – que le pipeline Spark ait terminé la transformation, avant de relire la base.

5.3.2 Choix techniques et limites

Cette architecture synchrone (la requête HTTP attend la fin du traitement asynchrone) a été un compromis délibéré pour la durée du projet : elle évite de construire une API de notification côté frontend (*polling* ou *websocket*), au prix d’un temps de réponse pouvant atteindre plusieurs dizaines de secondes lors d’un cache manqué. C’est un axe d’amélioration identifié pour la suite du projet (cf. section 7.4).

5.4 Frontend Next.js : un parcours guidé en trois étapes

5.4.1 Présentation et objectif

Le frontend (Next.js 16, App Router, environ 1 200 lignes de code) est volontairement réduit à une seule route, organisée comme un assistant à états (`'city-select' | 'matches' | 'flights' | 'confirmation'`), animé par `framer-motion`. Il consomme l’API via un client minimal, sans bibliothèque de *data fetching* :

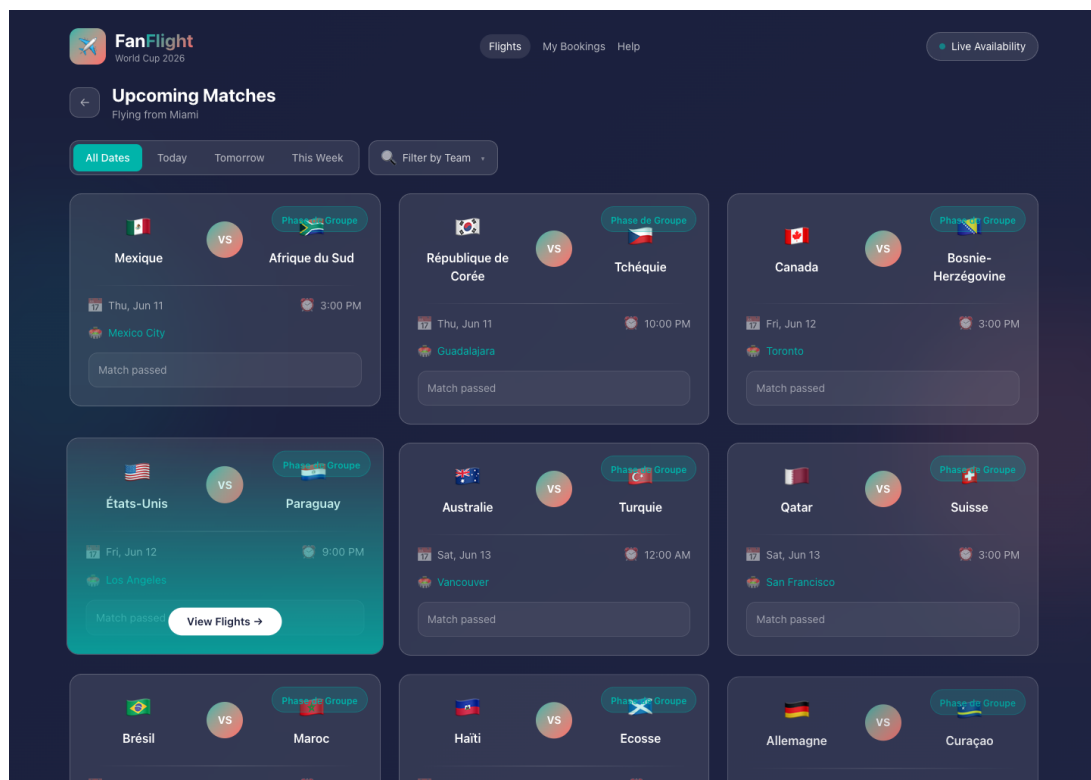
Listing 5.6 – frontend/src/lib/api.ts – client API minimal

```
const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000'

export const fetchFlights = async (matchId: number, departureCity: string) => {
  const res = await fetch(
    `${API_URL}/api/flights/${matchId}?departure_city=${encodeURIComponent(
      departureCity
    )}`
  );
  if (!res.ok) throw new Error('Erreur vols');
  return res.json();
};
```

5.4.2 Composants principaux

- **CitySelector** : sélection de la ville de départ, regroupée par pays (États-Unis, Mexique, Canada);
- **MatchFilters** / **MatchCard** : filtrage des matchs par équipe et par période, affichage du calendrier;
- **FlightCard** : présentation d'une proposition de vol (segments, correspondances, prix);
- **Header** : bandeau applicatif.

FIGURE 5.2 – Écran des matchs disponibles (**MatchFilters** + **MatchCard**) depuis Miami

L'interface repose sur Tailwind CSS 4 pour le style et ne nécessite aucune bibliothèque de gestion d'état externe, la complexité de l'application restant volontairement contenue pour un projet pédagogique mené à quatre.

5.5 Conteneurisation et intégration/déploiement continu

5.5.1 Orchestration locale

Listing 5.7 – `docker-compose.yml` – extrait, dépendances de démarrage

```
backend-api:
  build: ./API
  depends_on:
    db:
      condition: service_healthy

db:
  image: postgres:17
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U root -d ${DB_NAME}"]
```

L'image PostgreSQL est explicitement épinglée en version 17 plutôt que `latest` : un changement de version majeure non contrôlé pourrait rendre un volume de données existant incompatible sans migration explicite (`pg_upgrade`) – un risque jugé inacceptable pour une base de production.

5.5.2 Intégration continue : construction et publication des images

Le workflow GitHub Actions `frontend-deploy.yml` se déclenche sur chaque *push* vers la branche principale touchant le frontend, l'API ou le pipeline Spark, ou manuellement (`workflow_dispatch`). Il construit et publie trois images vers Docker Hub, chacune taguée à la fois `latest` et par le SHA du commit pour garantir la traçabilité :

Listing 5.8 – Extrait du workflow – publication taguée par SHA

```
- name: Build and push backend API image
  uses: docker/build-push-action@v6
  with:
    context: ./API
    tags: |
      ${ env.BACKEND_API_IMAGE }:latest
      ${ env.BACKEND_API_IMAGE }:${ github.sha }
```

5.5.3 Déploiement continu : SSH, Docker Compose et Watchtower

Le second job du workflow se connecte en SSH à l'instance de production, génère un fichier `docker-compose.yml` de production (ajoutant un service `watchtower` et des étiquettes `com.centurylinklabs.watchtower.enable=true` sur chaque service applicatif), puis démarre ou met à jour les services. Watchtower assure ensuite la mise à jour continue des conteneurs en production, en interrogeant Docker Hub toutes les 60 secondes, sans nouvelle intervention du pipeline CI :

Listing 5.9 – Service Watchtower – mise à jour continue en production

```
watchtower:
  image: containrrr/watchtower:latest
  command: --interval 60 --cleanup --label-enable
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
```

5.6 Observabilité de l'infrastructure : Prometheus et Grafana

5.6.1 Présentation et objectif

En complément du socle applicatif, Emy Regna a conçu et déployé, sur la même branche que l'extension machine learning, une chaîne d'observabilité complète de l'infrastructure FanFlight : supervision système, applicative et réseau, restituée dans un tableau de bord Grafana unique. Ce travail n'est pas encore fusionné dans la branche de production, mais démontre concrètement le volet « monitoring » de la compétence BC03 « Automatiser le déploiement de nouvelles versions de la solution et son monitoring ».

5.6.2 Une chaîne de collecte à plusieurs niveaux

La stack repose sur Prometheus comme collecteur central (scrutation toutes les 15 secondes), alimenté par cinq sources complémentaires, chacune couvrant un niveau différent de l'infrastructure :

- **node-exporter** : métriques système de l'hôte (CPU, mémoire, disque) de l'instance EC2 de production ;
- **cAdvisor** : métriques par conteneur (CPU, mémoire), nécessitant un accès privilégié en lecture au socket Docker et au système de fichiers hôte ;
- **postgres-exporter** : métriques internes de la base PostgreSQL, connecté via la même chaîne de connexion applicative que l'API ;
- **Blackbox Exporter** : *health-checks* HTTP actifs sur le frontend, la route `/ready` de l'API, et les interfaces web de Spark master et worker ;
- **Prometheus lui-même** : auto-supervision de sa propre disponibilité.

Listing 5.10 – `monitoring/prometheus/prometheus.yml` – *health-checks* HTTP via Blackbox Exporter

```
- job_name: service-health
  metrics_path: /probe
  params:
    module: [http_2xx]
  static_configs:
    - targets:
      - http://frontend:3000/
      - http://backend-api:8000/ready
      - http://spark-master:8080/
      - http://spark-worker:8081/
  relabel_configs:
    - source_labels: [__address__]
      target_label: __param_target
    - source_labels: [__param_target]
      target_label: instance
    - target_label: __address__
      replacement: blackbox-exporter:9115
```

Cette configuration illustre un motif classique de Blackbox Exporter : les cibles à sonder sont déclarées comme de simples URLs, puis réaffectées (*relabeling*) pour être effectivement interrogées via l'exporter plutôt que directement – Prometheus scrute ainsi la disponibilité de chaque service applicatif sans que celui-ci ait besoin d'exposer lui-même un point de métrique dédié.

5.6.3 Restitution : un tableau de bord Grafana provisionné

Grafana est connecté à Prometheus via une source de données provisionnée automatiquement au démarrage (fichier YAML, sans configuration manuelle dans l'interface), de même que le tableau de bord *FanFlight Overview*, dont les sept panneaux couvrent les trois niveaux de supervision de la stack :

Panneau	Niveau	Source
Service Health Checks	Applicatif	Blackbox Exporter (<code>/probe</code>)
HTTP Probe Duration	Applicatif	Blackbox Exporter
Exporter Health	Infrastructure de monitoring	Prometheus (<i>self-scrape</i> de chaque exporter)
EC2 CPU Usage	Système (hôte)	node-exporter
EC2 Memory Usage	Système (hôte)	node-exporter
EC2 Root Disk Usage	Système (hôte)	node-exporter
Container CPU Usage	Conteneur	cAdvisor

5.6.4 Intégration au déploiement

La chaîne d'observabilité est packagée dans `docker-compose.local.yml` pour les tests, et son déploiement en production est intégré au workflow GitHub Actions : les fichiers de configuration sont copiés vers `/opt/fanflight/monitoring/` sur l'instance EC2, et les identifiants d'administration Grafana sont paramétrables par variable d'environnement plutôt que codés en dur. Comme pour l'extension machine learning, cette intégration n'est aujourd'hui présente que sur la branche de développement dédiée ; sa fusion vers la branche principale est identifiée comme une priorité de court terme (cf. section 7.4).

5.7 Extension Machine Learning : prédiction de retard de vol

5.7.1 Présentation et objectif

En parallèle du socle décrit ci-dessus, Daniel Kouacou a développé, sur une branche dédiée du dépôt, une extension complète de machine learning visant à afficher, pour chaque vol proposé à un supporter, une probabilité de retard. Ce travail n'est pas encore fusionné dans la branche de production, mais constitue une démonstration à part entière de la compétence BC02 « développer un algorithme d'intelligence artificielle ».

5.7.2 Un problème de cohérence des données résolu par une source alternative

La première tentative de modélisation entraînait le modèle sur les vols 2026 renvoyés par SerpAPI, croisés avec une météo 2025 non alignée temporellement – un défaut méthodologique documenté explicitement par son auteur. La solution retenue sépare proprement les deux flux :

- **entraînement** : données réelles de ponctualité des vols domestiques américains publiées par le *Bureau of Transportation Statistics* (BTS) pour mai, juin et juillet 2025, filtrées aux aéroports des villes hôtes de la Coupe du Monde, enrichies de la météo réelle du jour et de l'aéroport concernés (Open-Meteo) – garantissant une cohérence temporelle complète entre le retard observé et la météo du jour ;
- **inférence** : les vols 2026 renvoyés par SerpAPI sont scorés en temps réel par le modèle ainsi entraîné, la météo utilisée à l'inférence provenant d'une prévision réelle si le vol a lieu dans les 16 jours, ou d'un repli saisonnier sinon.

5.7.3 Un contrat de features partagé entre entraînement et inférence

Pour éviter toute divergence entre les données vues à l'entraînement et celles vues en production (*train/serve skew*), le module `feature_builder.py` (environ 165 lignes) est partagé entre le pipeline d'entraînement et l'API de service, garantissant un encodage strictement identique des 20 caractéristiques utilisées (horaires, durée, compagnie, aéroports, et sept variables météorologiques dont un score de risque dérivé du code météo).

5.7.4 Entraînement, suivi et mise en service via MLflow

Le script `train.py` (environ 440 lignes) entraîne et compare deux modèles – XGBoost et un réseau de neurones PyTorch – et journalise métriques, graphiques (SHAP, courbe ROC, matrice de confusion) et artefacts dans MLflow, avant de promouvoir le meilleur modèle au statut « Production » dans le registre de modèles. Au démarrage, l'API charge automatiquement la version la plus récente enregistrée dans le registre pour ce modèle – qui correspond en pratique à la version promue, puisque chaque entraînement promeut la dernière version :

Listing 5.11 – Extrait de `main.py` (branche ML) – chargement du modèle au démarrage

```
@app.on_event("startup")
def load_model():
    global MODEL, MAPPINGS, MODEL_NAME
    try:
        client = MlflowClient()
        versions = client.get_latest_versions("flight_delay_xgboost")
        latest = max(versions, key=lambda v: int(v.version))
        local_path = client.download_artifacts(latest.run_id, "model/model.xgb")
        booster = xgb.Booster()
        booster.load_model(local_path)
        MODEL = booster
    except Exception as e:
        print(f"[ML] Echec chargement XGBoost : {e}")
        print("[ML] API démarre sans modele - vols retournes sans score de retard.")
```

Ce chargement échoue de façon contrôlée – l'API démarre sans modèle plutôt que de planter – si aucun modèle n'a encore été entraîné et promu, ce qui constitue une limite assumée : le déploiement n'est pas encore auto-suffisant sans une exécution manuelle préalable du pipeline d'entraînement.

5.7.5 Résultats et limites assumées

Sur 628 704 vols réels (contre 246 dans l'approche initiale fondée sur des données 2026 simulées), le taux de retard observé est de 27,8 %, et le modèle XGBoost retenu atteint une AUC de 0,728 (contre 0,713 pour le réseau de neurones). Plusieurs limites sont explicitement documentées par l'auteur : le prix n'étant pas disponible dans les données BTS, cette variable est neutralisée à l'entraînement ; la durée de vol est approximée par la distance, faute de donnée directe ; et l'entraînement ne porte que sur des vols directs, les correspondances n'étant pas couvertes par le jeu de données BTS.

5.7.6 Monitoring de dérive

Le script `drift_monitor.py` (environ 355 lignes) compare la distribution des variables d'un nouveau lot de données à celle du jeu de référence via des tests statistiques (indice de stabilité de population – PSI – et test de Kolmogorov-Smirnov), posant les bases d'une supervision de la qualité du modèle en production dans la durée – une pratique de MLOps mature, même si l'alerting automatique n'est pas encore implémenté.

6. Description d’une situation de travail ayant nécessité une recherche

6.1 Contexte de la problématique

Dès la phase de conception de FanFlight, une contrainte s’est imposée comme structurante pour l’ensemble de l’architecture : l’API externe de vols (SerpAPI) facture chaque requête effectuée. Avec un budget personnel limité pour ce projet pédagogique, une architecture naïve – interroger SerpAPI à chaque consultation d’un utilisateur, pour chaque combinaison de ville de départ et de match – aurait rapidement épuisé notre quota, sans même garantir une expérience utilisateur rapide (chaque appel externe prenant plusieurs secondes).

Le problème à résoudre était donc double : ne jamais payer deux fois pour la même information, tout en construisant un système qui reste correct lorsque l’information n’est, par nature, pas immédiatement disponible sous une forme exploitable (une réponse SerpAPI est un document JSON complexe, pas une ligne de table prête à être filtrée en SQL).

6.2 Démarche de recherche

Plusieurs pistes ont été évaluées avant de converger vers la solution retenue :

- **Cache côté frontend (navigateur)** : rejeté, car il ne mutualise rien entre utilisateurs différents interrogeant le même trajet, ce qui ne réduit pas le nombre d’appels à l’échelle de l’application ;
- **Cache mémoire côté API** : rejeté, car il serait perdu à chaque redémarrage de conteneur et ne serait pas partagé entre plusieurs instances de l’API si l’application devait être répliquée ;
- **Cache dédié (Redis)** : envisagé, mais écarté pour la durée du projet : il aurait ajouté un composant d’infrastructure supplémentaire à opérer et à sécuriser, pour un bénéfice marginal par rapport à une solution s’appuyant sur une base déjà présente dans l’architecture ;
- **Réutilisation de PostgreSQL comme cache durable** : retenue. La base de données, déjà nécessaire pour stocker le calendrier des matchs et les données de référence, pouvait aussi bien servir de mémoire persistante des résultats de vols déjà obtenus.

Cette dernière option soulevait cependant une difficulté : une réponse SerpAPI est un document JSON imbriqué, non directement interrogeable par une clause `WHERE` SQL utile (filtrer par ville, par date, par segment). Il fallait donc, en plus du cache lui-même, concevoir une étape de transformation entre la donnée brute reçue et la donnée requêteable.

6.3 Implémentation de la solution

La solution mise en œuvre combine trois éléments complémentaires :

1. une table d’atterrissage `FLIGHTS_RAW` qui reçoit la réponse SerpAPI telle quelle, sous forme JSONB, sans bloquer sur sa transformation ;

2. un pipeline de transformation asynchrone (Spark, déclenché par LISTEN/NOTIFY) qui structure cette donnée brute en lignes exploitables dans `FACT_FLIGHT`, sans quoi elle resterait inutilisable par une simple requête SQL ;
3. une fonction de lecture (`check_flight_db`) systématiquement appelée *avant* tout appel à SerpAPI, transformant l'appel externe en un recours de dernier ressort plutôt qu'un point de passage systématique.

Un outil complémentaire, `replay_job.py`, exécuté automatiquement à chaque démarrage du conteneur API, permet de réinjecter dans le pipeline des réponses SerpAPI déjà obtenues et sauvegardées sur disque, sans consommer de nouveau quota d'API – un choix directement motivé par la même contrainte de coût. Ce script n'est cependant pas idempotent : chaque redémarrage du conteneur déclenche un nouveau job de traitement, ce qui constitue une limite identifiée pour une exécution répétée en production (cf. section 7.4).

6.4 Itération ultérieure : un registre explicite des trajets déjà interrogés

Le prolongement du projet par Daniel Kouacou (branche machine learning) a affiné cette stratégie de cache en documentant l'introduction d'une table `QUERIED_ROUTES`, destinée à enregistrer chaque trajet interrogé – y compris lorsque SerpAPI n'a renvoyé *aucun* vol – avec un paramètre explicite (`force=true`) pour forcer un rafraîchissement volontaire (tests, données jugées obsolètes). Cette évolution, décrite dans la documentation du projet, corrige une limite identifiée de la première version : sans registre des trajets *sans résultat*, un trajet effectivement sans vol disponible risquait d'être réinterrogé indéfiniment, consommant du quota pour ne jamais recevoir de réponse utile. À l'état actuel de la branche, cette table n'est cependant plus présente dans le code après une fusion intermédiaire ayant résolu `main.py` en faveur d'une autre branche parente – un exemple concret de la difficulté à maintenir la cohérence entre documentation et code sur une branche de longue durée non encore intégrée à la production, et un point de vigilance retenu pour la fusion définitive de cette extension (cf. section 7.4).

Cette itération illustre bien la nature incrémentale d'une problématique de coût en ingénierie de données : la première solution (cache positif via `FACT_FLIGHT`) a résolu le cas dominant, avant qu'un usage plus poussé du système ne révèle un angle mort (le cas négatif) que la solution suivante a cherché à traiter.

7. Conclusion et ouverture

7.1 Synthèse du projet et réponse aux problématiques

Le projet FanFlight visait à démontrer la maîtrise d’une démarche data engineering complète, de la modélisation des données jusqu’à la mise en production automatisée, au service d’un besoin concret : simplifier la planification de déplacements autour de la Coupe du Monde 2026. Au-delà de l’enjeu applicatif, l’objectif était de couvrir l’intégralité des compétences du référentiel RNCP38919 – Data Engineer.

Résultats obtenus :

- une application fonctionnelle en production, permettant de croiser calendrier de matchs et propositions de vols ;
- un modèle de données hybride (schéma en étoile + JSONB) adapté à une donnée externe volatile ;
- un pipeline ETL événementiel, idempotent, déclenché par notification PostgreSQL plutôt que par ordonnancement périodique ;
- une architecture explicitement conçue pour maîtriser le coût d’un service externe payant ;
- une chaîne CI/CD complète (GitHub Actions, Docker Hub, déploiement SSH, mise à jour continue par Watchtower) ;
- une chaîne d’observabilité de l’infrastructure (Prometheus, Grafana) couvrant les niveaux système, conteneur et applicatif ;
- une extension machine learning aboutie (XGBoost, MLflow, monitoring de dérive), démontrant la compétence d’intégration d’un algorithme d’intelligence artificielle dans une chaîne de données.

7.2 Difficultés rencontrées et solutions apportées

Coût de l’API externe de vols. SerpAPI facturant chaque requête, une architecture naïve n’était pas soutenable. *Solution* : conception d’un cache durable porté par PostgreSQL, complété par un registre explicite des trajets déjà interrogés (cf. chapitre 6).

Incertitude sur l’hébergement de production. Aucune infrastructure de production n’était garantie au démarrage du projet. *Solution* : développement et validation intégrale en local (Docker Compose) avant tout déploiement réel, ce qui a permis de ne pas bloquer l’avancement technique en attendant une solution d’hébergement ; celle-ci a finalement été apportée par notre école (Liora), qui a mis à disposition une instance AWS EC2 en fin de projet.

Coordination d’une équipe distribuée et contrainte par des alternances. Les quatre membres de l’équipe, répartis sur toute la France et engagés chacun dans une alternance en entreprise, ne pouvaient compter sur un temps de travail synchrone important. *Solution* : discipline de revue de code asynchrone via des *pull requests* GitHub, découpage du travail en lots intégrables indépendamment, et accompagnement régulier de notre mentor Dan Cohen pour garder le cap malgré l’absence de coprésence.

7.3 Principaux bénéfices tirés du projet

Montée en compétences techniques :

- conception de modèles de données hybrides combinant schéma en étoile et données semi-structurées ;
- construction d'un pipeline de données événementiel avec Apache Spark et les primitives natives de PostgreSQL ;
- conteneurisation et automatisation complète d'une chaîne de déploiement (CI/CD, mise à jour continue) ;
- initiation concrète aux pratiques de MLOps (registre de modèles, monitoring de dérive) au travers de l'extension machine learning du projet.

Compétences collectives : la principale réussite de ce projet, au-delà de sa réalisation technique, a été de maintenir une collaboration efficace à quatre malgré nos contraintes organisationnelles (cf. chapitre 3) – une compétence directement transférable à un contexte professionnel de travail distribué.

7.4 Préconisations pour l'évolution du projet

Court terme :

- fusionner proprement l'extension machine learning et la chaîne d'observabilité Prometheus/Grafana dans la branche principale, en déplaçant les artefacts volumineux (jeux de données BTS, modèles) hors du dépôt Git vers un stockage dédié (versionnement de données, stockage objet), et en revalidant à cette occasion les fonctionnalités documentées mais perdues en cours de fusion (registre `QUERIED_ROUTES`) ;
- configurer des règles d'alerte Prometheus (Alertmanager) sur les métriques déjà collectées par Emy Regna (indisponibilité d'un service, saturation CPU/mémoire de l'instance EC2), pour transformer la supervision existante, aujourd'hui consultée manuellement dans Grafana, en une détection proactive des incidents ;
- remplacer l'attente synchrone de l'API pendant le traitement Spark par un mécanisme de notification asynchrone côté frontend, pour ne plus bloquer la requête HTTP jusqu'à 90 secondes ;
- rendre `replay_job.py` idempotent ou conditionnel à un premier démarrage, pour éviter de déclencher un nouveau job Spark à chaque redémarrage du conteneur API en production ;
- faire tourner les conteneurs API et Spark avec un utilisateur non privilégié, à l'image de ce qui est déjà fait pour le frontend ;
- ajouter la contrainte de clé étrangère entre `FACT_FLIGHT` et `DIM_AIRPORT` une fois `enrich_airports` déplacé avant le chargement des vols, pour que l'intégrité référentielle soit garantie par PostgreSQL plutôt que par la seule discipline applicative ;
- sécuriser les identifiants par défaut actuellement présents dans le code (mots de passe de secours codés en dur), au profit d'une gestion de secrets systématique.

Moyen terme :

- mettre en place des tests automatisés (unitaires sur les fonctions de transformation, d'intégration sur le pipeline complet) dans la chaîne CI/CD ;
- ajouter un système d'alerte sur les résultats du monitoring de dérive du modèle de prédiction de retard ;
- implémenter le parcours de confirmation/réservation, aujourd'hui présent dans l'interface mais non connecté à une logique métier.

Long terme :

- industrialiser le service du modèle de machine learning (service HTTP dédié plutôt que partage par volume Docker) ;
- étendre la couverture du calendrier aux phases finales de la compétition et généraliser le modèle de prédiction de retard au-delà des seuls vols directs.

7.5 Ouverture

Ce projet fil rouge a été, pour chacun des quatre membres de l'équipe, une mise en pratique concrète des compétences du métier de Data Engineer sur un cas d'usage complet – de la modélisation d'un entrepôt de données jusqu'à l'intégration d'un modèle de machine learning en production. Les enseignements en dépassent le cadre pédagogique : la nécessité de concevoir une architecture de données sous contrainte de coût dès sa conception, l'exigence de rigueur qu'impose un pipeline automatisé et idempotent, et la capacité à faire fonctionner une équipe technique malgré la distance géographique sont autant de compétences que nous comptons chacun continuer à approfondir dans nos parcours respectifs en ingénierie des données – que ce soit en consolidant nos pratiques d'architecture événementielle, en allant plus loin dans l'industrialisation de modèles de machine learning, ou en poursuivant l'exploration des pratiques de MLOps amorcées avec l'extension prédictive de FanFlight.

Annexes

Annexe A : Documentation officielle

FastAPI

- Documentation officielle : <https://fastapi.tiangolo.com/>
- Déploiement : <https://fastapi.tiangolo.com/deployment/>

PostgreSQL

- Documentation officielle : <https://www.postgresql.org/docs/>
- LISTEN/NOTIFY : <https://www.postgresql.org/docs/current/sql-notify.html>

Apache Spark / PySpark

- Documentation officielle : <https://spark.apache.org/docs/latest/api/python/>
- Spark SQL Functions : <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions.html>

Next.js

- Documentation officielle : <https://nextjs.org/docs>

Docker et Docker Hub

- Documentation officielle Docker : <https://docs.docker.com/>
- Docker Compose : <https://docs.docker.com/compose/>

GitHub Actions

- Documentation officielle : <https://docs.github.com/actions>

MLflow

- Documentation officielle : <https://mlflow.org/docs/latest/index.html>
- Model Registry : <https://mlflow.org/docs/latest/model-registry.html>

XGBoost

- Documentation officielle : <https://xgboost.readthedocs.io/>

Observabilité (Prometheus, Grafana)

- Prometheus – documentation officielle : <https://prometheus.io/docs/>
- Grafana – documentation officielle : <https://grafana.com/docs/>
- Blackbox Exporter (health-checks HTTP) : https://github.com/prometheus/blackbox_exporter
- cAdvisor (métriques par conteneur) : <https://github.com/google/cadvisor>

Annexe B : Sources de données externes

- SerpAPI (Google Flights) : <https://serpapi.com/google-flights-api>
- Open-Meteo : <https://open-meteo.com/>
- Bureau of Transportation Statistics (BTS), On-Time Performance : <https://www.transtats.bts.gov/>
- FIFA – Coupe du Monde 2026 : <https://www.fifa.com/fr/tournaments/mens/worldcup/canadamexicousa2026>

Annexe C : Blogs techniques et veille technologique

Ingénierie de données (blogs d'entreprise)

- AWS Big Data Blog : <https://aws.amazon.com/blogs/big-data/>
- Databricks Blog : <https://www.databricks.com/blog>
- Netflix Technology Blog : <https://netflixtechblog.com/>
- Uber Engineering Blog : <https://www.uber.com/blog/engineering/>

Écosystème PostgreSQL et Python

- Planet PostgreSQL (agrégateur de blogs officiels) : <https://planet.postgresql.org/>
- Real Python (FastAPI, Python applicatif) : <https://realpython.com/>

Architecture logicielle

- Martin Fowler – articles sur les patterns de données et d'architecture : <https://martinfowler.com/>

Annexe D : Communautés et forums

Forums techniques

- Stack Overflow (tag *apache-spark*) : <https://stackoverflow.com/questions/tagged/apache-spark>
- Stack Overflow (tag *fastapi*) : <https://stackoverflow.com/questions/tagged/fastapi>
- Stack Overflow (tag *postgresql*) : <https://stackoverflow.com/questions/tagged/postgresql>
- Stack Overflow (tag *mlflow*) : <https://stackoverflow.com/questions/tagged/mlflow>
- Stack Overflow (tag *grafana*) : <https://stackoverflow.com/questions/tagged/grafana>

Reddit

- r/dataengineering : <https://www.reddit.com/r/dataengineering/>
- r/PostgreSQL : <https://www.reddit.com/r/PostgreSQL/>
- r/docker : <https://www.reddit.com/r/docker/>

Slack et Discord

- MLOps Community Slack : <https://mlops.community/>
- dbt Community Slack (pratiques de transformation de données) : <https://www.getdbt.com/community/>

GitHub Discussions

- Discussions des projets open-source mobilisés (FastAPI, Next.js, MLflow, Prometheus, Grafana)

Annexe E : Outils et utilitaires

- DBeaver / pgAdmin (exploration de la base PostgreSQL) : <https://dbeaver.io/>
- Docker Desktop (environnement de développement local) : <https://www.docker.com/products/docker-desktop/>
- Documentation OpenAPI interactive de FastAPI (/docs), pour tester l'API sans outil externe
- GitHub CLI (gh), pour piloter les *pull requests* et les *workflows* GitHub Actions en ligne de commande : <https://cli.github.com/>
- dbdiagram.io, pour esquisser et faire évoluer le schéma en étoile : <https://dbdiagram.io/>

Annexe F : Certifications et formations

- Fiche RNCP38919 – Data engineer (France Compétences) : <https://www.francecompetences.fr/>
- DataScientest – formation Data Engineer : <https://datascientest.com/data-engineer-rncp>
- AWS Certified Data Engineer – Associate : <https://aws.amazon.com/certification/certified-data-engineer-associate/>
- Databricks Certified Data Engineer Associate : <https://www.databricks.com/learn/certification/data-engineer-associate>

Annexe G : Ressources complémentaires

Livres recommandés

- *Fundamentals of Data Engineering* – Joe Reis, Matt Housley (O'Reilly, 2022)
- *Designing Data-Intensive Applications* – Martin Kleppmann (O'Reilly, 2017)
- *The Data Warehouse Toolkit* – Ralph Kimball, Margy Ross – référence directe pour la conception du schéma en étoile de FanFlight

Podcasts

- Data Engineering Podcast (Tobias Macey) : <https://www.dataengineeringpodcast.com/>
- The Data Stack Show : <https://datastackshow.com/>

Conférences

- Data Council : <https://www.datacouncil.ai/>
- dbt Coalesce (dbt Labs) : <https://coalesce.getdbt.com/>